Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas
**Docentes:** Edgar Francisco Rodas Robledo, Herman Igor Véliz Linares
**Tutores Académicos:** Emiliano José Alexander Velásquez Najera, Josue Daniel Chavez Portillo, Jorge Andrés Mejía Súchite

# Pensar como programador

## Tiempo estimado: 10 min

## Fecha programada: Semana 5 – 15/08/2026

| | |
|---|---|
| **Título del Documento / Capítulo** | Pensar como programador: descomposición, patrones y abstracción |
| **Autor(es)** | Tutores Académicos – Laboratorio 768 Algoritmos (con base en Wing, 2006) |
| **Tipo de Recurso** | Documento Técnico |

Índice

1. Marco Formativo.....2
1.1 Valores.....2
1.2 Competencia(s).....2
2. Palabras Clave.....2
3. Introducción.....3
4. Desarrollo del Tema.....3
4.1. Análisis del problema.....3
4.2. Técnicas.....3
4.3. De la técnica al algoritmo.....3
5. Orientación a Casos de Estudio.....3
5.1. Problema Real.....3
5.2. Solución Aplicada.....3
Figura 1 - Descomposición de un problema en subproblemas.....4
5.3. Lecciones Aprendidas.....4
Tabla 1 - Técnicas de pensamiento computacional.....4
6. Conclusiones.....5
7. Preguntas de Autoevaluación.....5
8. Referencias.....5

# 1. Marco Formativo

## 1.1 Valores

| Nombre del valor | ¿Cómo se aplica en tu laboratorio? |
|---|---|
| **Responsabilidad** | Dedicar el tiempo necesario a analizar el contexto, las entradas y las salidas del problema antes de diseñar la solución algorítmica. |

## 1.2 Competencia(s)

| Tipo de Competencia | |
|---|---|
| Competencia General | Analiza problemas computacionales y formula soluciones algorítmicas utilizando pensamiento computacional, pseudocódigo y diagramas de flujo en las tecnologías establecidas para representar el procesamiento y flujo de datos bajo criterios de corrección lógica, claridad y orden |
| Competencia Específica | Descompone problemas computacionales en subproblemas manejables empleando técnicas de descomposición, reconocimiento de patrones y abstracción para identificar entradas, procesos y salidas antes de diseñar la solución algorítmica |

# 2. Palabras Clave

**Pensamiento computacional:** Proceso de resolución de problemas apoyado en conceptos de la computación.

**Descomposición:** Dividir un problema en partes más pequeñas.

**Patrón:** Similitud que se repite entre problemas o subproblemas.

**Abstracción:** Selección de lo esencial ignorando detalles.

**Instancia:** Caso concreto de un problema.

# 3. Introducción

Antes de escribir una sola línea de pseudocódigo, un programador analiza el problema. Esa habilidad se denomina pensamiento computacional y fue popularizada por Jeannette Wing en 2006.

La lectura presenta las técnicas básicas y cómo se aplican en un caso real.

# 4. Desarrollo del Tema

## 4.1. Análisis del problema

Se identifican el contexto, las entradas, los procesos, las salidas y las restricciones. Un problema bien definido es claro, relevante y tiene una necesidad real de solución.

## 4.2. Técnicas

La descomposición divide el problema; el reconocimiento de patrones reutiliza soluciones; la abstracción elimina detalles irrelevantes; la generalización permite resolver todas las instancias.

## 4.3. De la técnica al algoritmo

Con el problema descompuesto, cada subproblema se convierte en un bloque del algoritmo, y los patrones repetitivos suelen traducirse en ciclos.

# 5. Orientación a Casos de Estudio

## 5.1. Problema Real

**Descripción del Contexto:** Un grupo de estudiantes recibe el encargo de «hacer un sistema para la cafetería». Comienzan a programar de inmediato sin definir qué datos necesitan ni qué resultados espera la cafetería.

**Impacto:** A mitad del trabajo descubren que faltan datos y que el programa no resuelve la necesidad real: se pierde tiempo y se rehace trabajo.

## 5.2. Solución Aplicada

**Marco Teórico / Metodología:** Se aplica el pensamiento computacional: primero se define el problema y se descompone, luego se reconocen patrones y se generaliza.

**Pasos de Implementación:**

- Entrevistar al usuario y escribir entradas, procesos, salidas y restricciones.
- Descomponer en subproblemas (menú, ventas, inventario, reportes).
- Reconocer patrones comunes (registrar, buscar, listar) y generalizar.

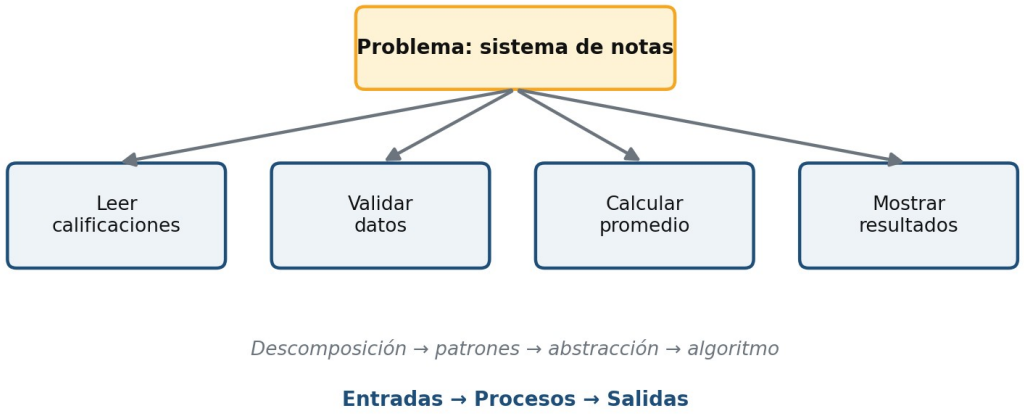



**Figura 1 - Descomposición de un problema en subproblemas**

## 5.3. Lecciones Aprendidas

**Resultados Clave:** El diseño es claro y el trabajo se reparte de forma ordenada.

**Consideraciones / Limitaciones:** Una descomposición excesiva puede complicar la integración; debe equilibrarse.

| Técnica | Qué hace | Resultado |
|---|---|---|
| Descomposición | Divide el problema | Subproblemas simples |
| Patrones | Busca similitudes | Reutilización |
| Abstracción | Elimina detalles | Modelo esencial |
| Generalización | Amplía la solución | Válida para muchas instancias |

**Tabla 1 - Técnicas de pensamiento computacional**

## 6. Conclusiones

- Analizar antes de programar evita rehacer trabajo.
- Las cuatro técnicas se complementan.
- Todo algoritmo comienza con un problema bien definido.

## 7. Preguntas de Autoevaluación

1. ¿Cuál es el factor principal que causó el retrabajo en el caso descrito?
2. Explique con sus palabras la diferencia entre descomposición y abstracción.
3. Dé un ejemplo de patrón en dos problemas distintos.
4. ¿Qué significa generalizar una solución?
5. ¿Cómo aplicaría estas técnicas en su próxima práctica?

## 8. Referencias

| Tipo de Recurso | Descripción del Contenido | Enlace / Ubicación |
|---|---|---|
| Documentación Oficial (Si aplica) | Wing, J. M. (2006). Computational Thinking. Communications of the ACM | https://dl.acm.org/doi/10.1145/1118178.1118215 |
| Libro de Texto (Si aplica) | Polya, G. (1945). How to Solve It | Libro |
| Esquemas / Diagramas (Si aplica) | Figura 1 (elaboración propia) | Lectura S5 |
| Repositorio de Código (Si aplica) | Material del curso | UEDI |